Python Case Study Solutions Practical Record

10 case studies - 4 easy, 4 medium, 2 hard

Selected from the prescribed list of 100 Python problems.

Each case includes the statement, objectives, assumptions, algorithm, flowchart, pseudocode, source code, test cases, sample output, and conclusion.

Selected cases

Easy: #27 Advanced Numeric Pattern Generator

Easy: #13 ATM Transaction Sequence Simulation

Easy: #29 Password Policy Validation Engine

Easy: #35 Text Compression Using Character Runs

Medium: #14 Warehouse Inventory Reconciliation

Medium: #23 Student Timetable Conflict Detection

Medium: #31 String-Based Examination Answer Evaluation

Medium: #47 Sports Tournament Ranking System

Hard: #42 Student Marks Matrix Analysis

Hard: #86 Multi-Stage ATM Transaction Audit

Case 1: Advanced Numeric Pattern Generator

Prescribed list #27 | Difficulty: Easy

Problem statement

Generate a centered hollow diamond using a positive odd width. Reject invalid widths.

Objectives

Validate the dimension; use loops and conditionals to print a symmetric hollow diamond; keep the outline aligned.

Assumptions

Width is an odd integer from 3 to 15. The center row is the widest. A hollow row prints stars only at its two edges, except the top and bottom.

Algorithm

1. Read width w.
2. If w is less than 3, greater than 15, or even, report invalid input and stop.
3. For each row from 0 to w-1, compute its distance from the center.
4. Compute leading spaces and star positions; print the hollow row.
5. Stop.

Pseudocode

```
READ w
IF w < 3 OR w > 15 OR w MOD 2 = 0 THEN PRINT "Invalid width"
ELSE
  FOR row = 0 TO w-1
    spaces = ABS(center-row)
    PRINT spaces; stars at boundary positions
  END FOR
END IF
```

Python Source Code

Case 1: Advanced Numeric Pattern Generator | prescribed list #27

```
width = int(input("Enter an odd width from 3 to 15: "))


if width < 3 or width > 15 or width % 2 == 0:
    print("Invalid width")
else:
    middle = width // 2

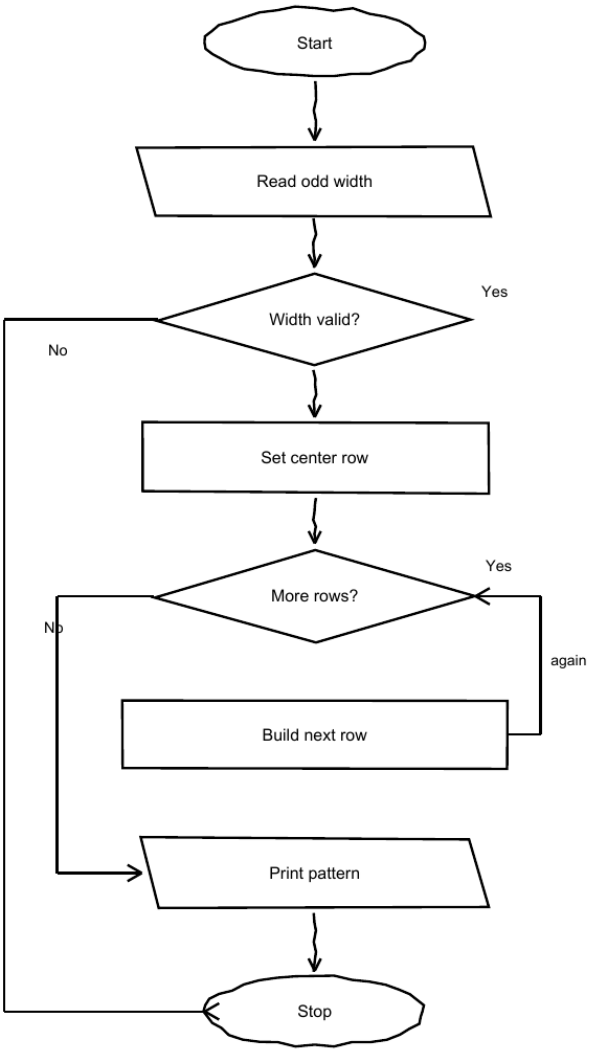

    for row in range(width):
        if row <= middle:
            level = row
        else:
            level = width - 1 - row


        if level == 0:
            print(" " * middle + "*")
        else:
            inside_spaces = 2 * level - 1
            print(" " * (middle - level) + "*" +
                  " " * inside_spaces + "*")
```

Flowchart

Flowchart - Advanced Numeric Pattern Generator

Case #27 | Easy



Black pen flowchart - hand-ruled symbols and arrows

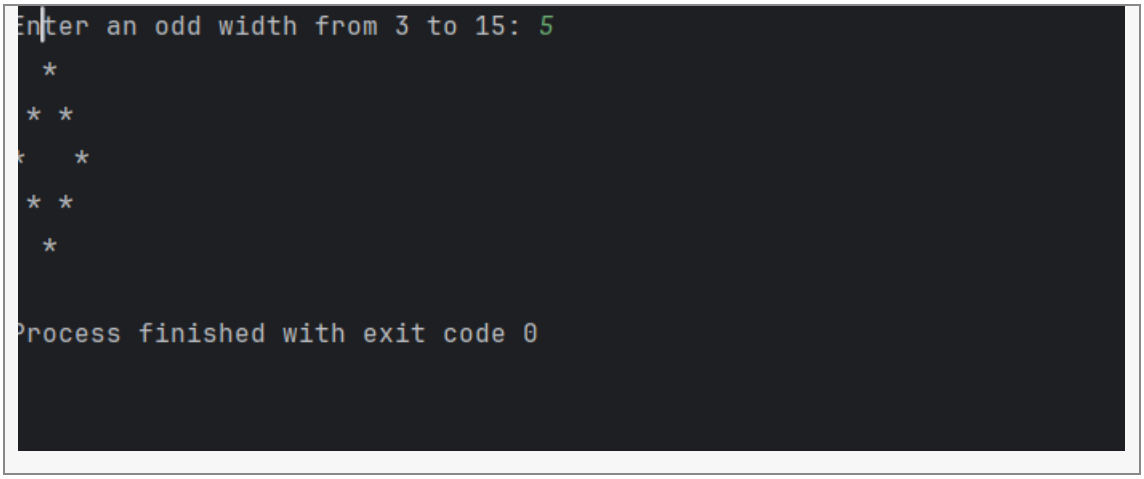
Test Cases and Output

Case #27 | Advanced Numeric Pattern Generator | Easy

Test cases

Input width	Expected result
5	Hollow diamond with 5 rows; widest row has 2 edge stars.
4	Invalid width
2	Invalid width

Sample output capture



Conclusion

The program validates an odd dimension and uses a loop with symmetry to print the hollow diamond.

The program checks that the width is a valid positive odd number, then uses loops to print a centred hollow diamond. Invalid widths are rejected with an error message.

Case 2: ATM Transaction Sequence Simulation

Prescribed list #13 | Difficulty: Easy

Problem statement

Simulate deposits and withdrawals in order, reject invalid transactions, enforce a withdrawal limit, and summarize accepted and rejected activity.

Objectives

Maintain a running balance; validate each operation; report totals and final balance.

Assumptions

Opening balance is nonnegative. Each transaction is a tuple (type, amount). Amount must be positive. A withdrawal cannot exceed the per-transaction limit or available funds.

Algorithm

1. Set opening balance and zero the counters.
2. For each transaction, validate its amount.
3. For deposit, add amount; for withdrawal, check limit and balance before subtracting.
4. Record rejected operations without changing balance.
5. Print summary.

Pseudocode

```
balance = opening balance
FOR each (kind, amount)
  IF amount <= 0: reject
  ELSE IF kind = deposit: add to balance
  ELSE IF kind = withdrawal AND amount <= limit AND amount <= balance: subtract
  ELSE: reject
END FOR
PRINT totals and balance
```

Python Source Code

```
opening_balance = 1000
withdrawal_limit = 500

# Each transaction is a tuple: type and amount
transactions = [
    ("D", 250),   # Deposit
    ("W", 300),   # Withdrawal
    ("W", 700),   # Rejected: over the limit
    ("D", -20)    # Rejected: amount must be positive
]

balance = opening_balance
deposits = 0
withdrawals = 0
rejected = 0

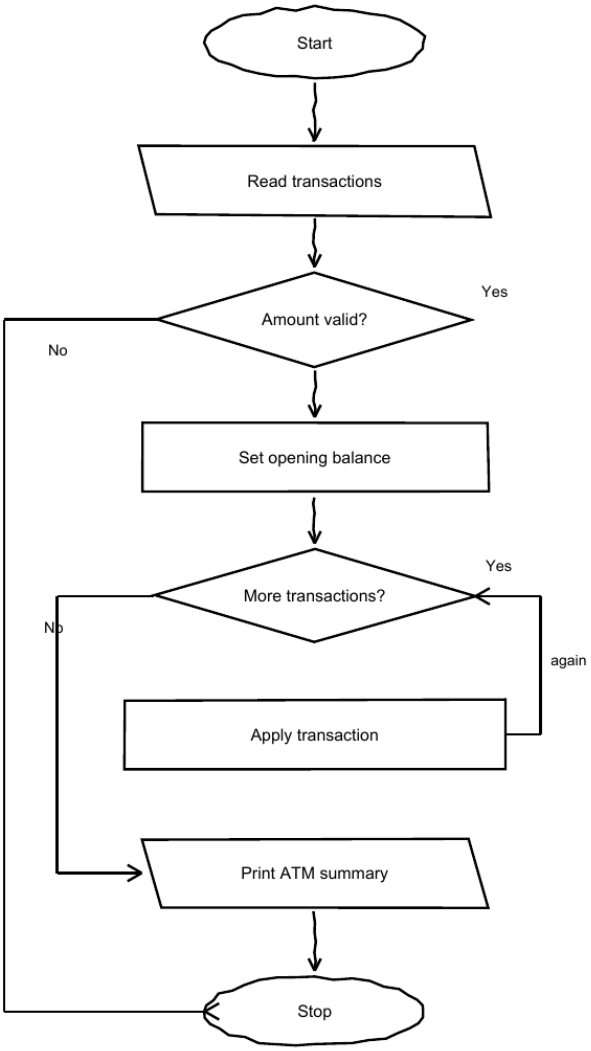
for kind, amount in transactions:
    if amount <= 0:
        rejected += 1
    elif kind == "D":
        balance += amount
        deposits += amount
    elif kind == "W" and amount <= withdrawal_limit and amount <= balance:
        balance -= amount
        withdrawals += amount
    else:
        rejected += 1

print("Accepted deposits:", deposits)
print("Accepted withdrawals:", withdrawals)
print("Rejected transactions:", rejected)
print("Final balance: $", balance, sep="")
```

Flowchart

Flowchart - ATM Transaction Sequence Simulation

Case #13 | Easy



Black pen flowchart - hand-ruled symbols and arrows

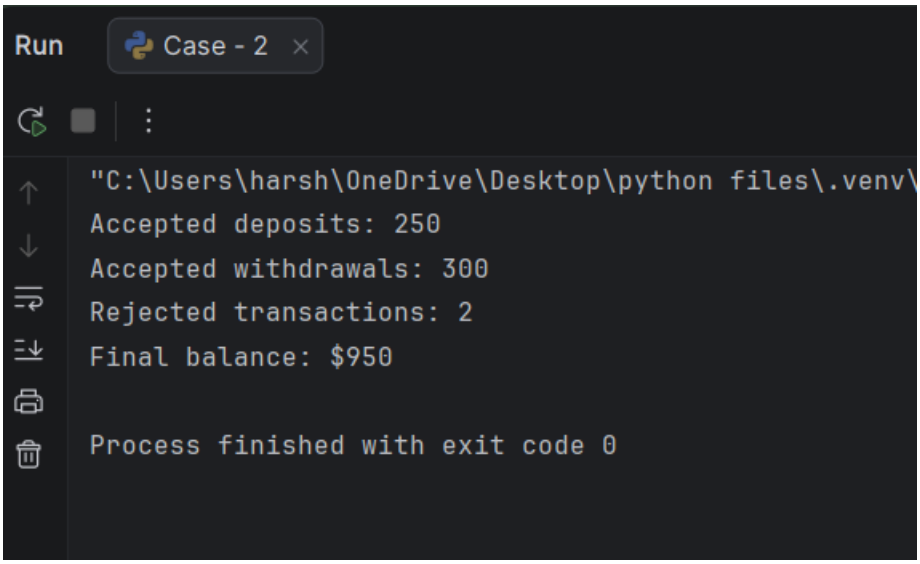
Test Cases and Output

Case #13 | ATM Transaction Sequence Simulation | Easy

Test cases

Transactions	Expected summary
D250, W300, W700, D-20	Deposits 250; withdrawals 300; rejected 2; balance 950
W100 from $50	Rejected; balance remains $50

Output



Conclusion :

The program processes deposits and withdrawals in order, rejects invalid transactions, and enforces the withdrawal limit. It then reports accepted totals, rejected transactions, and the final account balance.

Case 3: Password Policy Validation Engine

Prescribed list #29 | Difficulty: Easy

Problem statement

Evaluate sample passwords against minimum length, uppercase, lowercase, digit, special-character, and prohibited-substring rules. Explain each failed condition.

Objectives

Use string operations and conditionals to test every supplied rule and provide useful feedback.

Assumptions

A valid password has at least 8 characters, one uppercase, one lowercase, one digit, one of !@#$, and does not contain "python" in any case.

Algorithm

1. Read password.
2. Check each policy independently and collect failure messages.
3. If no failures, report valid; otherwise list all failures.

Pseudocode

```
READ password
errors = empty list
CHECK length, uppercase, lowercase, digit, special character, prohibited word
IF errors empty: PRINT valid
ELSE PRINT each error
```

Python Source Code

Case 3: Password Policy Validation Engine | prescribed list #29

```
password = input("Enter a password: ")
errors = []

has_uppercase = False
has_lowercase = False
has_digit = False
has_special = False

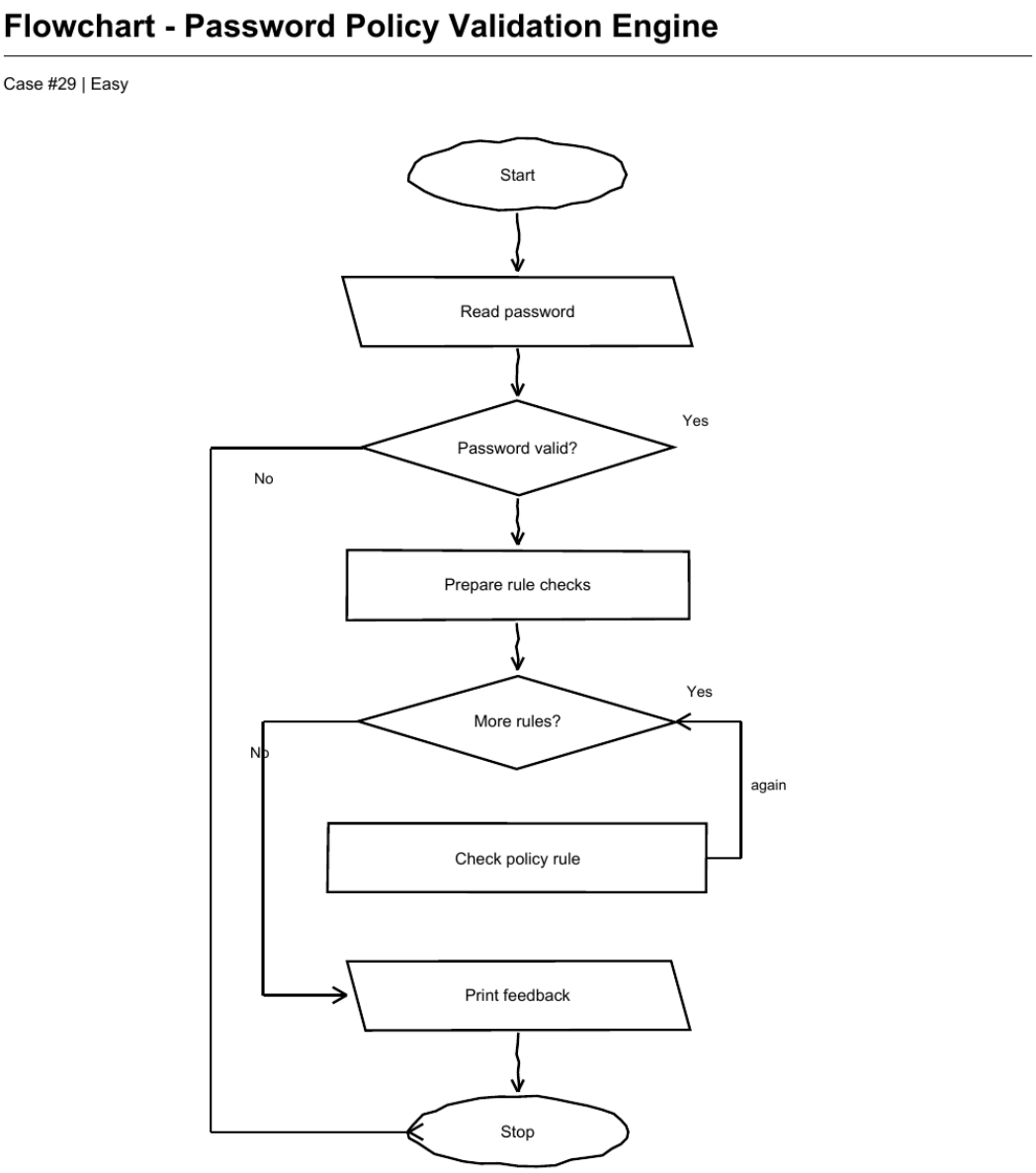
for character in password:
    if character.isupper():
        has_uppercase = True
    if character.islower():
        has_lowercase = True
    if character.isdigit():
        has_digit = True
    if character in "!@#$":
        has_special = True

if len(password) < 8:
    errors.append("It must contain at least 8 characters.")
if not has_uppercase:
    errors.append("It must contain an uppercase letter.")
if not has_lowercase:
    errors.append("It must contain a lowercase letter.")
if not has_digit:
    errors.append("It must contain a digit.")
if not has_special:
    errors.append("It must contain a special character: !, @, #, or $.")
if "python" in password.lower():
    errors.append("It must not contain the word 'python'.")

if len(errors) == 0:
    print("Password accepted.")
else:
```

```
    print("Password rejected:")
    for error in errors:
        print("-", error)
```

Flowchart



Test Cases and Output

Case #29 | Password Policy Validation Engine | Easy

Test cases

Password	Expected result
Abcdef1!	Password accepted
python1	Length, uppercase, special-character and prohibited-word failures

Sample output capture

Case - 3.py

```python
password = input("Enter a password: ")
errors = []

has_uppercase = False
has_lowercase = False
has_digit = False
has_special = False

for character in password:
    if character.isupper():
        has_uppercase = True
    if character.islower():
        has_lowercase = True
    if character.isdigit():
        has_digit = True
    if character in "!@#$":
        has_special = True

if len(password) < 8:
    errors.append("It must contain at least 8 characters.")
if not has_uppercase:
    errors.append("It must contain an uppercase letter.")
if not has_lowercase:
    errors.append("It must contain a lowercase letter.")
if not has_digit:
    errors.append("It must contain a digit.")
if not has_special:
    errors.append("It must contain a special character: !, @, #, or $.")
if "python" in password.lower():
    errors.append("It must not contain the word 'python'.")

if len(errors) == 0:
    print("Password accepted.")
else:
```

Conclusion

Every policy is checked separately, allowing the user to correct multiple problems in one attempt.

Case 4: Text Compression Using Character Runs

Prescribed list #35 | Difficulty: Easy

Problem statement

Encode consecutive repeated characters as character-count pairs, then report whether the encoded form is shorter.

Objectives

Traverse a string, count adjacent runs, build an encoding, and compare lengths.

Assumptions

Input contains letters only and is nonempty. Encoding format is character followed by run length, including 1.

Algorithm

1. Read text and initialize encoding.
2. Scan each character; count matching consecutive characters.
3. Append character and count; continue at the next run.
4. Compare encoded and original lengths and print result.

Pseudocode

```
READ text
i = 0; encoded = empty string
WHILE i < length(text)
  count consecutive copies of text[i]
  append text[i] and count
  advance i by count
END WHILE
PRINT encoded and length comparison
```

Python Source Code

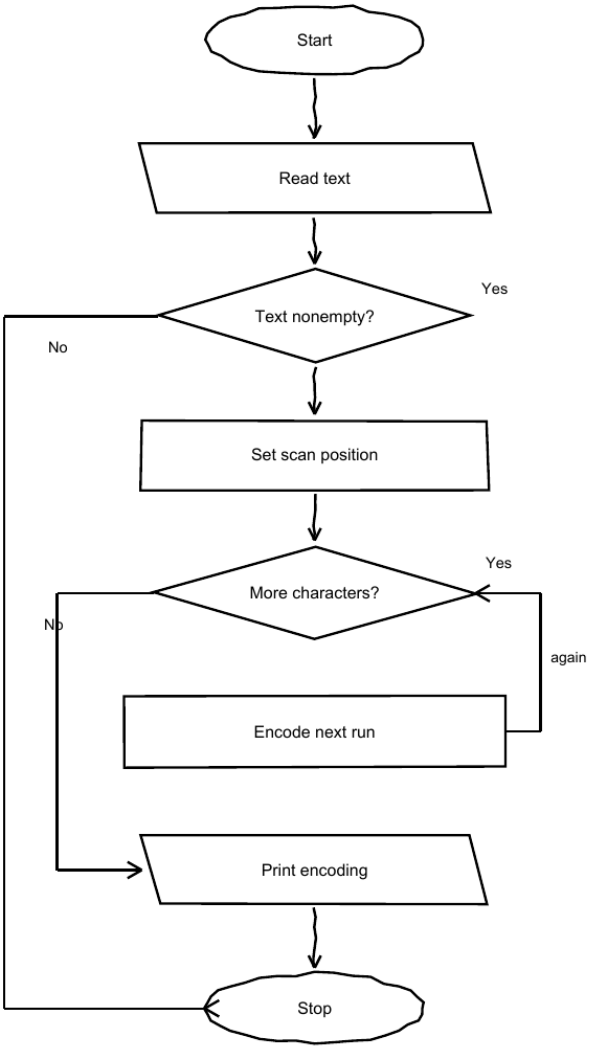
Case 4: Text Compression Using Character Runs | prescribed list #35

```
text = input("Text: ")
encoded = ""
i = 0
while i < len(text):
    count = 1
    while i + count < len(text) and text[i + count] == text[i]:
        count += 1
    encoded += text[i] + str(count)
    i += count
print("Encoded:", encoded)
print("Original length:", len(text))
print("Encoded length:", len(encoded))
if len(encoded) < len(text):
    print("Encoding is beneficial")
else:
    print("Keep original text")
```

Flowchart

Flowchart - Text Compression Using Character Runs

Case #35 | Easy



Black pen flowchart - hand-ruled symbols and arrows

Test Cases and Output

Case #35 | Text Compression Using Character Runs | Easy

Test cases

Text	Expected result
aaabbc	a3b2c1; encoded length 6, keep original
aaaaaaaa	a8; encoded length 2, beneficial

Sample output capture



Conclusion :

Nested repetition checks identify runs correctly; the length comparison prevents using an encoding that expands the data.

Case 5: Warehouse Inventory Reconciliation

Prescribed list #14 | Difficulty: Medium

Problem statement

Compare recorded opening, incoming, dispatched, and counted closing stock for products; flag discrepancies and calculate replenishment to a target.

Objectives

Use parallel lists and loops to compute expected closing stock, discrepancy, and required restock.

Assumptions

All quantities are nonnegative integers. Expected closing = opening + incoming - dispatched. Counted stock is physical stock. Target stock is 20 units.

Algorithm

1. Store products and quantity records in parallel lists.
2. For each product calculate expected closing and discrepancy.
3. If physical stock is below target, calculate replenishment.
4. Display one reconciliation row per product.

Pseudocode

```
FOR each product i
 expected = opening[i] + incoming[i] - dispatched[i]
 difference = physical[i] - expected
 replenish = MAX(0, target - physical[i])
 PRINT product, expected, difference, replenish
END FOR
```

Python Source Code

Case 5: Warehouse Inventory Reconciliation | prescribed list #14

```
products = ["Pens", "Books", "Folders"]
opening_stock = [40, 25, 18]
incoming_stock = [10, 12, 5]
dispatched_stock = [32, 30, 8]
counted_closing_stock = [17, 8, 16]

target_stock = 20

print("Product  Expected  Counted  Difference  Replenish")

for i in range(len(products)):
    expected = opening_stock[i] + incoming_stock[i] - dispatched_stock[i]
    difference = counted_closing_stock[i] - expected

    if counted_closing_stock[i] < target_stock:
        replenish = target_stock - counted_closing_stock[i]
    else:
        replenish = 0

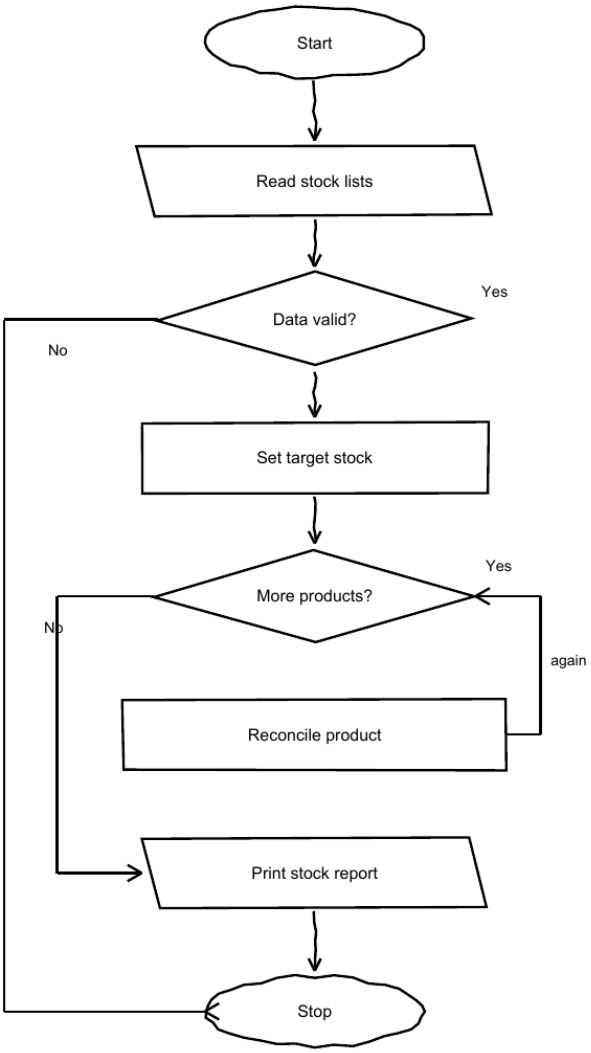
    print(
        f"{products[i]:8} "
        f"{expected:8} "
        f"{counted_closing_stock[i]:8} "
        f"{difference:10} "
        f"{replenish:10}"
    )

    if difference != 0:
        print("  Discrepancy detected")
    else:
        print("  Stock matches the records") 
```

Flowchart

Flowchart - Warehouse Inventory Reconciliation

Case #14 | Medium



Black pen flowchart - hand-ruled symbols and arrows

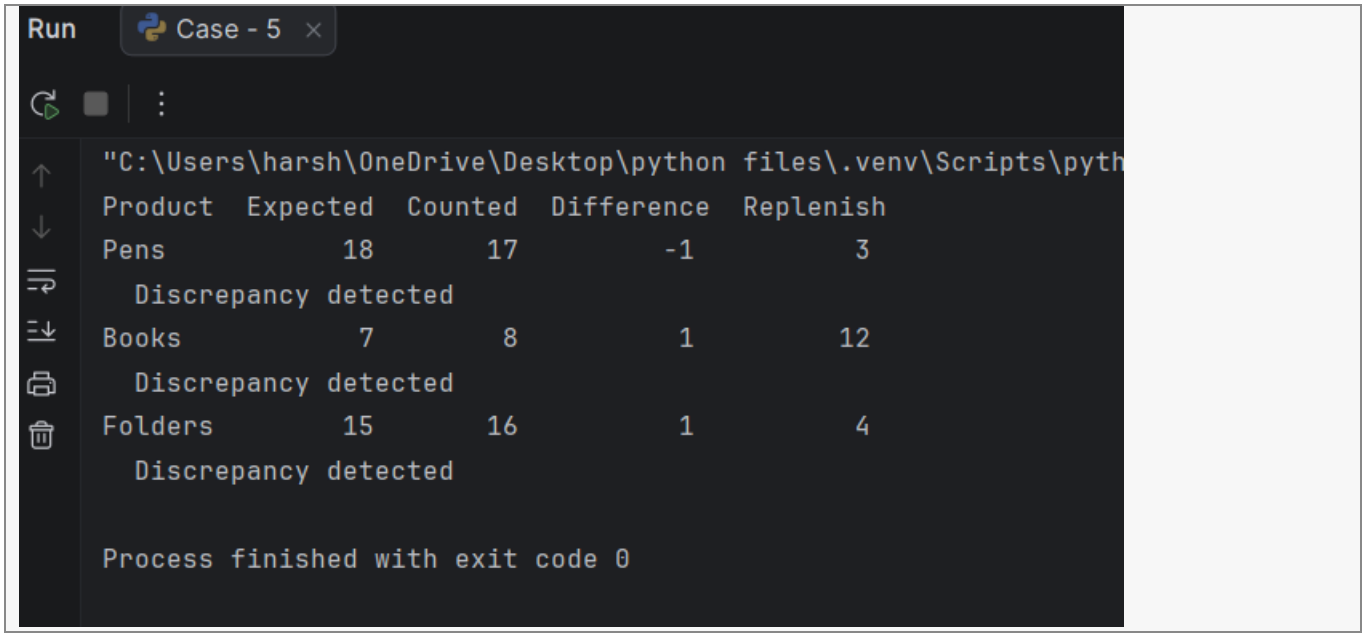
Test Cases and Output

Case #14 | Warehouse Inventory Reconciliation | Medium

Test cases

Product	Expected / difference / replenish
Pens	18 / -1 / 3
Books	7 / +1 / 12
Folders	15 / +1 / 4

Sample output capture



Conclusion

The reconciliation identifies both stock-count differences and replenishment quantities against a fixed target.

Case 6: Student Timetable Conflict Detection

Prescribed list #23 | Difficulty: Medium

Problem statement

Compare course periods represented as tuples and detect overlapping times for the same room or student.

Objectives

Apply interval-overlap logic to tuple data and report each conflicting pair.

Assumptions

Each class tuple is (course, student_group, room, start_hour, end_hour). Periods are half-open intervals; end equals another start is not a conflict.

Algorithm

1. Read class tuples.
2. Compare every unique pair.
3. Check whether their intervals overlap.
4. If they overlap and share a room or group, report the conflict.
5. If none are found, report conflict-free.

Pseudocode

```
FOR i from 0 to n-1
 FOR j from i+1 to n-1
  overlap = start_i < end_j AND start_j < end_i
  IF overlap AND (room_i = room_j OR group_i = group_j)
    PRINT conflict pair
END FOR
```

Python Source Code

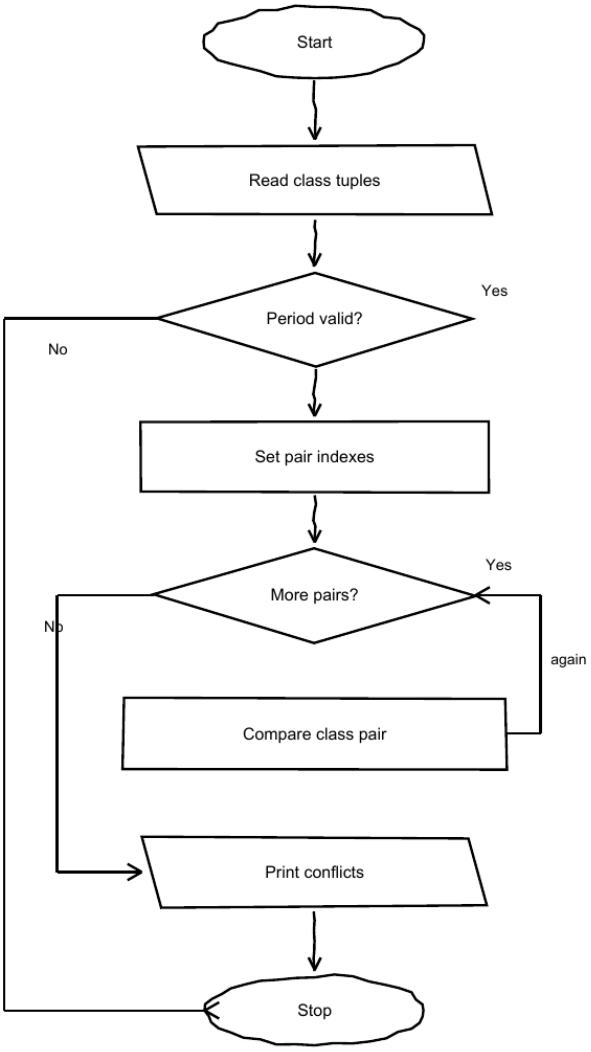
Case 6: Student Timetable Conflict Detection | prescribed list #23

```
classes = [("Math", "G1", "R1", 9, 10),
           ("Physics", "G1", "R2", 9, 11),
           ("History", "G2", "R1", 10, 11)]
found = False
for i in range(len(classes)):
    course_a, group_a, room_a, start_a, end_a = classes[i]
    for j in range(i + 1, len(classes)):
        course_b, group_b, room_b, start_b, end_b = classes[j]
        overlap = start_a < end_b and start_b < end_a
        shared = room_a == room_b or group_a == group_b
        if overlap and shared:
            print(course_a, "conflicts with", course_b)
            found = True
if not found:
    print("No timetable conflicts")
```

Flowchart

Flowchart - Student Timetable Conflict Detection

Case #23 | Medium



Black pen flowchart - hand-ruled symbols and arrows

Test Cases and Output

Case #23 | Student Timetable Conflict Detection | Medium

Test cases

Classes	Expected result
Math G1 R1 9-10; Physics G1 R2 9-11	Math conflicts with Physics
History G2 R1 10-11	No clash with Math (touching periods)

Sample output capture



Conclusion

Pairwise interval comparison detects clashes for shared rooms or groups while treating touching boundaries as non-overlapping.

Case 7: String-Based Examination Answer Evaluation

Prescribed list #31 | Difficulty: Medium

Problem statement

Compare student answer strings with a key, count correct, incorrect, and unanswered questions, apply negative marking, and calculate a score.

Objectives

Normalize letter case, compare corresponding answers, and compute a transparent final result.

Assumptions

Answer key contains A-D. Blank means unanswered. Correct = +2, incorrect = -0.5, unanswered = 0. Input list length must match the key.

Algorithm

1. Set answer key and scoring values.
2. Read answers and check list length.
3. For each position count correct, wrong, or blank.
4. Calculate score and print counts.

Pseudocode

```
IF answer count != key length: report invalid
ELSE
 FOR each answer position
  IF blank: unanswered++
  ELSE IF upper(answer) = key: correct++
  ELSE: incorrect++
 score = 2*correct - 0.5*incorrect
 PRINT summary
END IF
```

Python Source Code

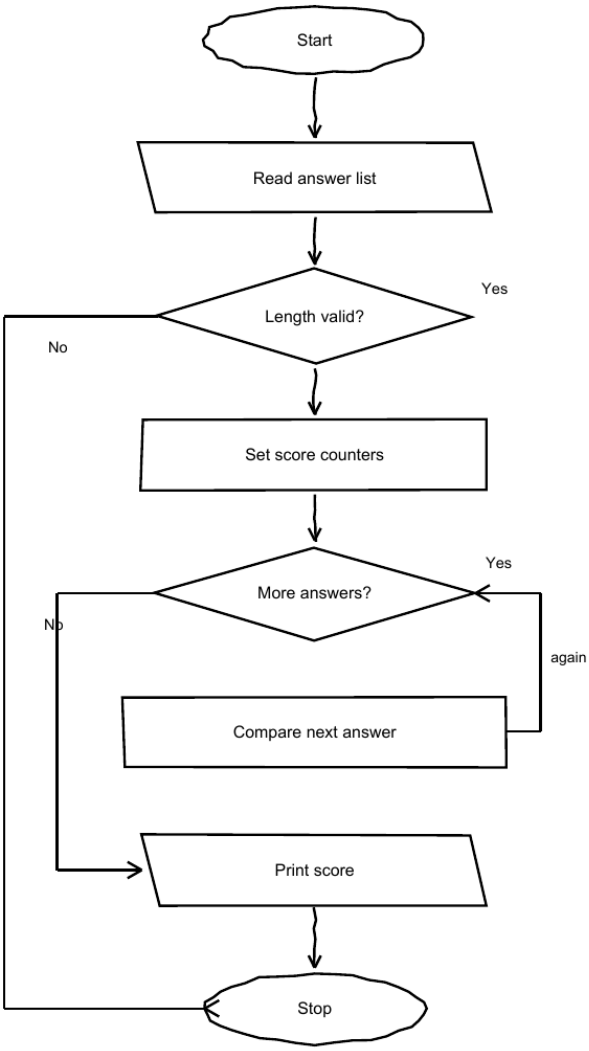
Case 7: String-Based Examination Answer Evaluation | prescribed list #31

```
key = ["A", "C", "B", "D", "A"]
answers = ["a", "B", "", "D", "C"]
if len(answers) != len(key):
    print("Invalid answer count")
else:
    correct = incorrect = blank = 0
    for i in range(len(key)):
        response = answers[i].upper()
        if response == "":
            blank += 1
        elif response == key[i]:
            correct += 1
        else:
            incorrect += 1
    score = 2 * correct - 0.5 * incorrect
    print("Correct:", correct)
    print("Incorrect:", incorrect)
    print("Unanswered:", blank)
    print("Score:", score)
```

Flowchart

Flowchart - String-Based Examination Answer Evaluation

Case #31 | Medium



Black pen flowchart - hand-ruled symbols and arrows

Test Cases and Output

Case #31 | String-Based Examination Answer Evaluation | Medium

Test cases

Answers	Expected result
a, B, blank, D, C	2 correct; 2 incorrect; 1 unanswered; 3.0 points
Only 4 answers	Invalid answer count

Sample output capture

```
Correct: 2
Incorrect: 2
Unanswered: 1
Score: 3.0

Process finished with exit code 0
```

Conclusion

Case-insensitive comparison and explicit blank handling produce a reliable result with the stated negative-marking policy.

Case 8: Sports Tournament Ranking System

Prescribed list #47 | Difficulty: Medium

Problem statement

Store team match outcomes, calculate points and goal difference, then rank teams using points followed by goal difference.

Objectives

Aggregate match results in dictionaries and use deterministic tie-breaking rules.

Assumptions

Win earns 3 points, draw 1, loss 0. Each result is (team A, team B, goals A, goals B). If points and goal difference tie, alphabetical order is used.

Algorithm

1. Initialize points and goal difference for each team.
2. For each match update goal differences and points.
3. Build a ranking list.
4. Sort by points descending, goal difference descending, then name ascending.
5. Print standings.

Pseudocode

```
INITIALIZE points and goal_difference for all teams
FOR each match
 update goal differences
 award points by outcome
END FOR
SORT teams by (-points, -goal_difference, name)
PRINT standings
```

Python Source Code

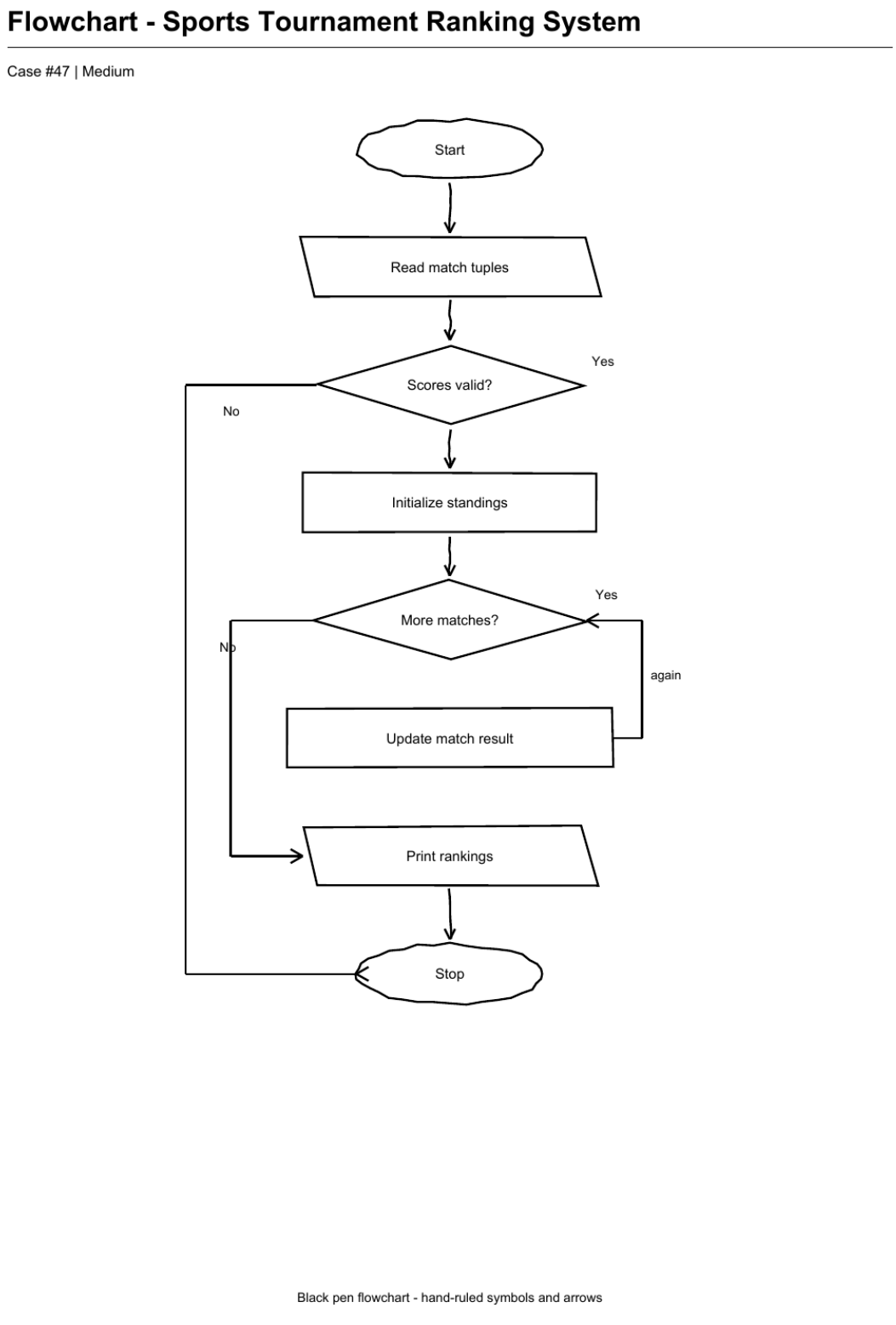
```
    ("Wolves", "Falcons", 3, 1)]
   teams = ["Falcons", "Tigers", "Wolves"]
points = [0, 0, 0]


   diff = [0, 0, 0]
   for a, b, ga, gb in matches:
       ia = ib = -1
       for i in range(len(teams)):
           if teams[i] == a:
               ia = i
           if teams[i] == b:
               ib = i
       diff[ia] += ga - gb
       diff[ib] += gb - ga
       if ga > gb:
           points[ia] += 3
       elif gb > ga:
           points[ib] += 3
       else:
           points[ia] += 1
           points[ib] += 1
   for i in range(len(teams)):
       for j in range(i + 1, len(teams)):
           swap = False
           if points[j] > points[i]:
               swap = True
           elif points[j] == points[i] and diff[j] > diff[i]:
               swap = True
           elif points[j] == points[i] and diff[j] == diff[i] and teams[j] < teams[i]:
               swap = True
           if swap:
               teams[i], teams[j] = teams[j], teams[i]
               points[i], points[j] = points[j], points[i]
               diff[i], diff[j] = diff[j], diff[i]
   for place in range(len(teams)):
       print(place + 1, teams[place], points[place], diff[place])
```

Case 8: Sports Tournament Ranking System | prescribed list #47

```
matches = [("Falcons", "Tigers", 2, 1),
           ("Tigers", "Wolves", 0, 0),
```

Flowchart



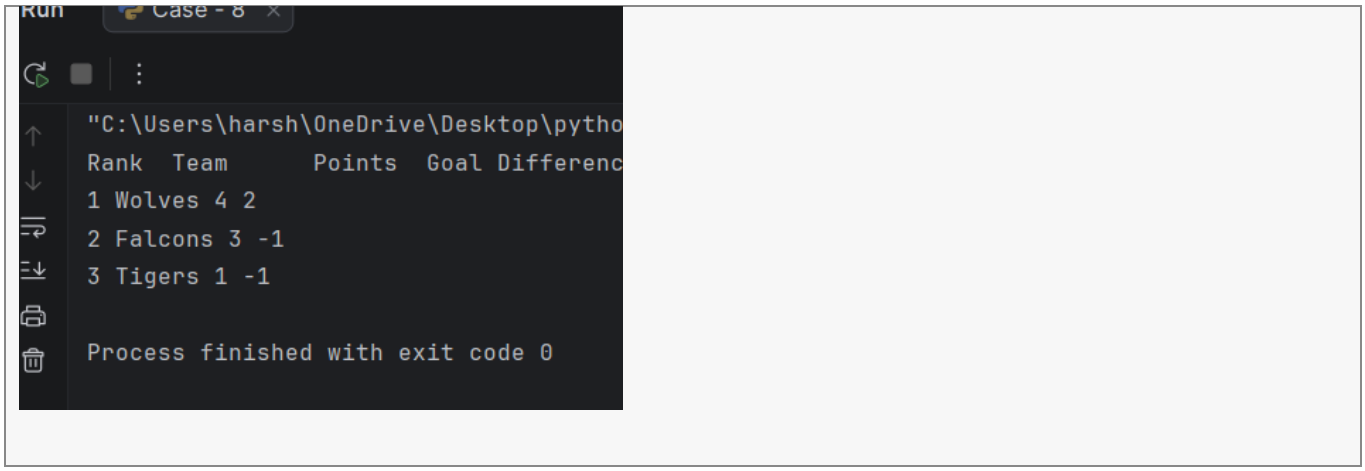
Test Cases and Output

Case #47 | Sports Tournament Ranking System | Medium

Test cases

Matches	Expected ranking
Falcons 2-1 Tigers; Tigers 0-0 Wolves; Wolves 3-1 Falcons	1 Wolves 4 +2; 2 Falcons 3 -1; 3 Tigers 1 -1

Sample output capture



Conclusion

The program produces repeatable standings by applying the declared points, goal-difference, and alphabetical tie-break rules.

Case 9: Student Marks Matrix Analysis

Prescribed list #42 | Difficulty: Hard

Problem statement

Analyze a matrix of students by subjects, calculate student totals and averages and subject averages, identify the top student and subjects below a threshold.

Objectives

Use nested lists and nested loops to calculate row and column statistics and apply threshold conditions.

Assumptions

Each row is one student and each column one subject. All marks are from 0 to 100. Student names and subject names match matrix dimensions. Low-average threshold is 50.

Algorithm

1. Store names and marks matrix.
2. For each student row calculate total and average.
3. For each subject column sum marks and calculate average.
4. Track the highest student average.
5. Flag subjects whose average is below 50.
6. Print the analysis.

Pseudocode

```
FOR each student row
 total = SUM(row); average = total / subject_count
 update highest performer
END FOR
FOR each subject column
 average = SUM(matrix[row][column]) / student_count
 IF average < 50: flag subject
END FOR
PRINT summaries
```

Python Source Code

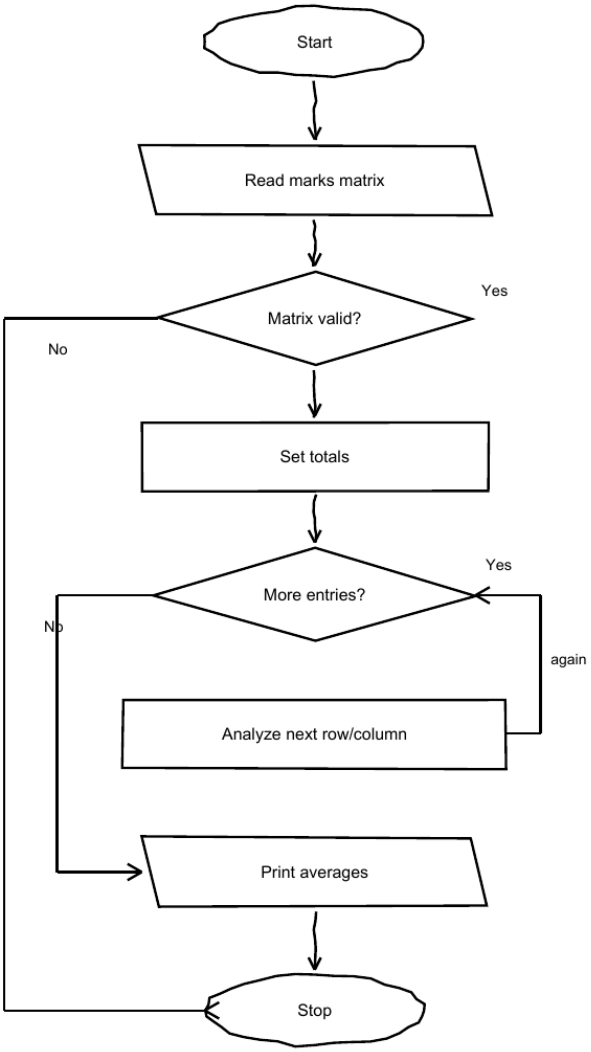
Case 9: Student Marks Matrix Analysis | prescribed list #42

```
students = ["Asha", "Bala", "Chen"]
subjects = ["Python", "Math", "English"]
marks = [[82, 76, 91], [48, 55, 62], [70, 68, 72]]
student_averages = []
for i in range(len(students)):
    total = 0
    for j in range(len(subjects)):
        total += marks[i][j]
    average = total / len(subjects)
    student_averages.append(average)
    print(students[i], "total", total, "average", round(average, 2))
top = 0
for i in range(1, len(student_averages)):
    if student_averages[i] > student_averages[top]:
        top = i
print("Top student:", students[top])
for j in range(len(subjects)):
    total = 0
    for i in range(len(students)):
        total += marks[i][j]
    average = total / len(students)
    if average < 50:
        status = "LOW"
    else:
        status = "OK"
    print(subjects[j], "average", round(average, 2), status)
```

Flowchart

Flowchart - Student Marks Matrix Analysis

Case #42 | Hard



Black pen flowchart - hand-ruled symbols and arrows

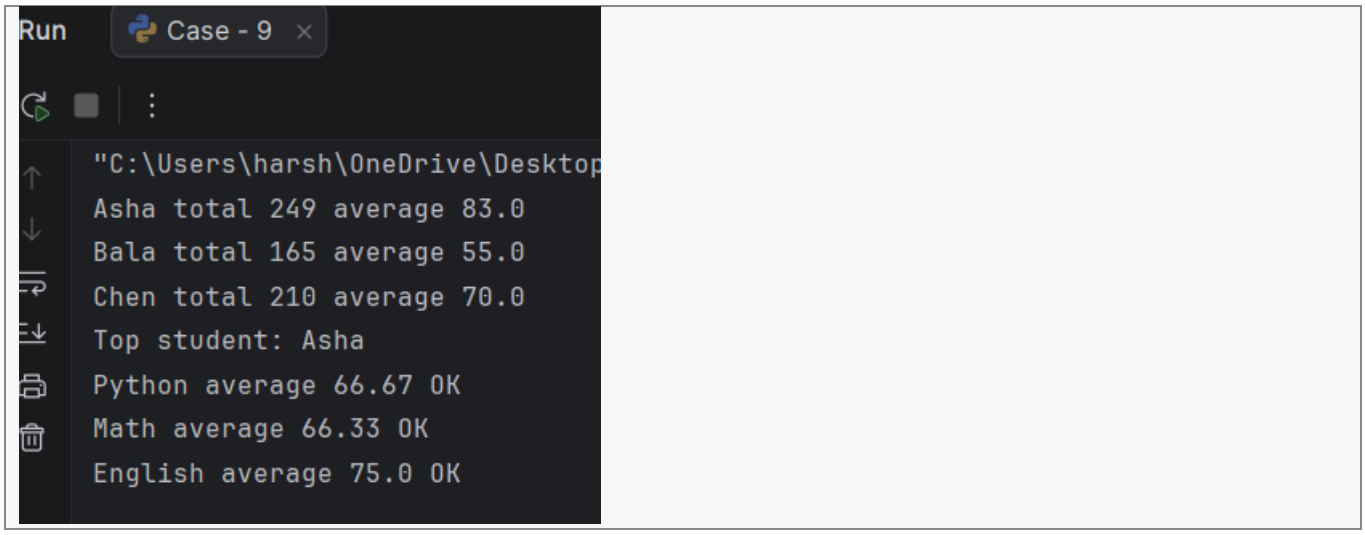
Test Cases and Output

Case #42 | Student Marks Matrix Analysis | Hard

Test cases

Matrix	Expected summary
[82,76,91]; [48,55,62]; [70,68,72]	Asha top avg 83.0; subject averages 66.67, 66.33, 75.0; no low subject
All marks 0	All averages 0; all subjects flagged LOW

Sample output capture



Conclusion

Nested loops provide both row-wise and column-wise analysis, while the threshold flag highlights subjects that may need attention.

Case 10: Multi-Stage ATM Transaction Audit

Prescribed list #86 | Difficulty: Hard

Problem statement

Process deposits, withdrawals, and transfers in sequence; maintain the balance, reject invalid operations, and flag repeated withdrawal amounts.

Objectives

Combine tuple unpacking, loops, conditionals, account limits, transaction totals, and repeated-pattern detection.

Assumptions

Opening balance $1,000. Deposits/withdrawals/transfers must be positive. Withdrawal cap $500; transfer cap $400. Transactions are (type, amount). Transfer is outgoing. Same withdrawal amount twice consecutively is suspicious.

Algorithm

1. Initialize balance, counters, prior withdrawal, and flags.
2. For each transaction tuple validate amount and type.
3. For withdrawal or transfer, enforce cap and available balance.
4. Update balance and totals only when accepted.
5. Flag consecutive repeated withdrawal amount.
6. Print audit summary.

Pseudocode

```
FOR each (kind, amount)
 IF amount <= 0: reject
 ELSE IF kind = D: add amount
 ELSE IF kind = W and within limit and funds: subtract
 ELSE IF kind = T and within limit and funds: subtract
 ELSE reject
 IF current W amount = previous W amount: flag repeated
END FOR
PRINT audit totals and flags
```

Python Source Code

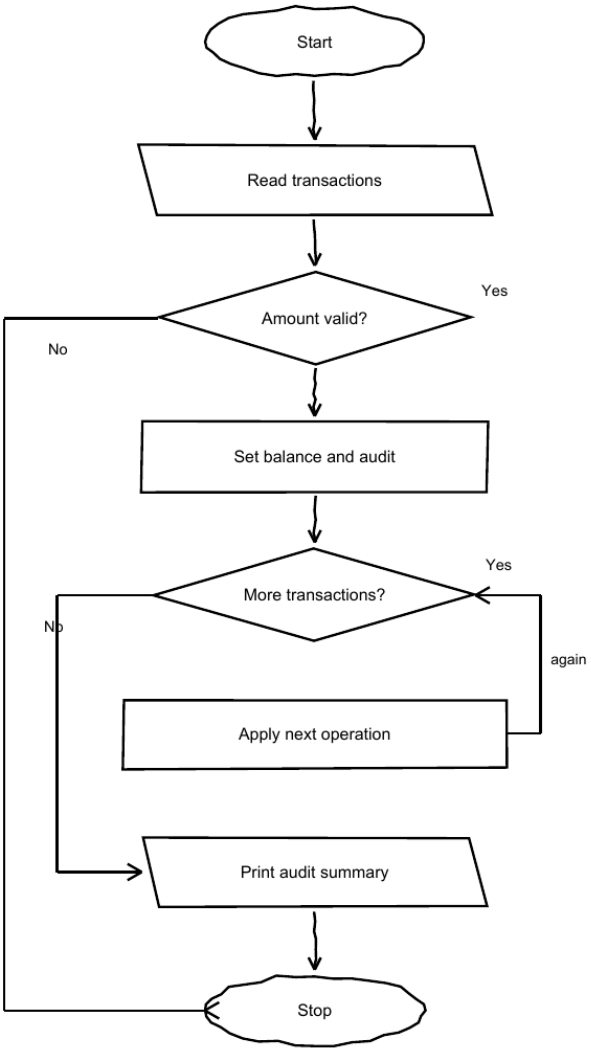
Case 10: Multi-Stage ATM Transaction Audit | prescribed list #86

```
transactions = [("D", 200), ("W", 300), ("W", 300),
                ("T", 250), ("W", 600), ("D", -5)]
balance = 1000
accepted = rejected = 0
withdrawn = transferred = deposited = 0
previous_withdrawal = None
repeated = False
for kind, amount in transactions:
    if kind != "W":
        previous_withdrawal = None
    valid = amount > 0
    if kind == "D" and valid:
        balance += amount
        deposited += amount
    elif kind == "W" and valid and amount <= 500 and amount <= balance:
        balance -= amount
        withdrawn += amount
        if previous_withdrawal == amount:
            repeated = True
        previous_withdrawal = amount
    elif kind == "T" and valid and amount <= 400 and amount <= balance:
        balance -= amount
        transferred += amount
        previous_withdrawal = None
    else:
        valid = False
    if valid:
        accepted += 1
    else:
        rejected += 1
        previous_withdrawal = None
print("Accepted:", accepted, "Rejected:", rejected)
print("Deposited:", deposited, "Withdrawn:", withdrawn,
      "Transferred:", transferred)
print("Final balance: $", balance, sep="")
print("Repeated withdrawal flag:", "YES" if repeated else "NO")
```

Flowchart

Flowchart - Multi-Stage ATM Transaction Audit

Case #86 | Hard



Black pen flowchart - hand-ruled symbols and arrows

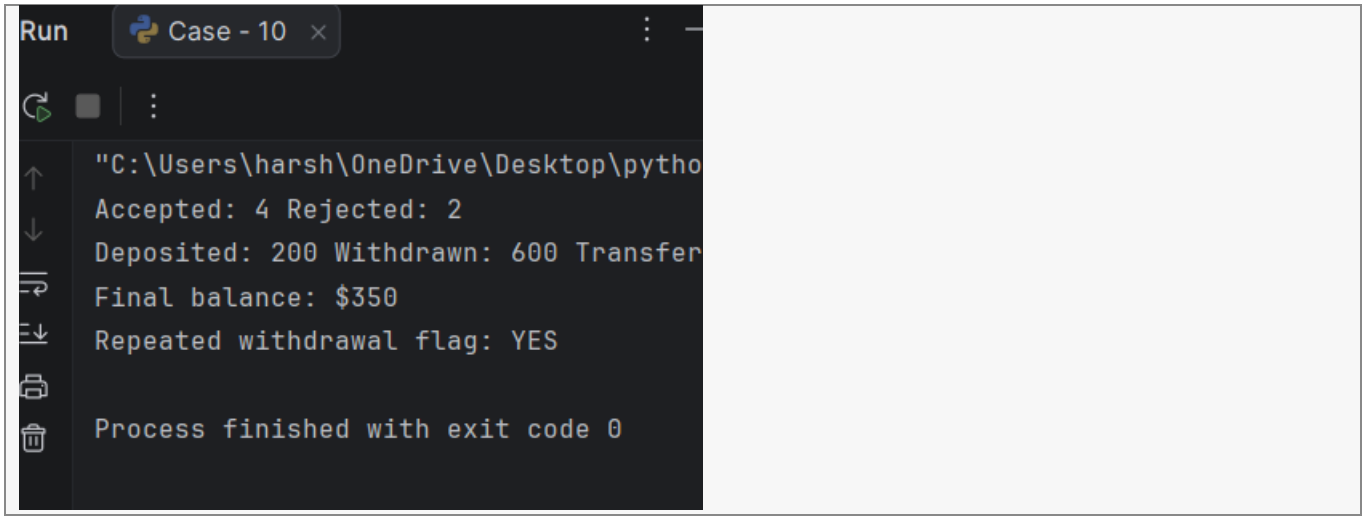
Test Cases and Output

Case #86 | Multi-Stage ATM Transaction Audit | Hard

Test cases

Sequence	Expected result
D200, W300, W300, T250, W600, D-5	4 accepted, 2 rejected; ending balance $350; repeated withdrawal YES
W600 from $1000	Rejected; balance stays $1000

Sample output capture



Conclusion

The audit updates balances only for accepted operations and identifies a repeated withdrawal pattern while retaining an auditable transaction summary.